Security Audit Report — PortalIT / LootIT

Date: 2026-04-28 · Source: external auditor vs. internal parallel-agent review · Status: action plan

Contents

1. Why my audit missed what it missed
2. Finding-by-finding report (10 items)
3. Risk-vs-effort matrix
4. Recommended fix order

Why my audit missed what it missed — root cause

My audit ran 4 parallel agents, each scoped to a directory (`src/` , `apps/lootit/` , `server/` , plus a generic security pass). The misses fall into three patterns:

1. **No cross-cutting "where do secrets live" pass.** Each agent looked at code in its directory. None of them did `SELECT column_name FROM information_schema WHERE column_name LIKE '%api_key%' OR '%secret%' OR '%token%'` then mapped those columns back to RLS policies. That's how the `spanning_api_key` exposure went unnoticed.
2. **Scope didn't include the entire repo.** I excluded `extensions/` , the `scripts/` audit tooling, and didn't cross-reference frontend `client.entities.*` calls against migration files. That's how INKY token storage, the incomplete RLS audit script, and `lootit_contracts` schema drift slipped through.
3. **Generic flags without file:line citations.** My security agent said "watch for open redirects" but didn't grep `returnUrl|window.location.href` . The auditor pointed to `Login.jsx:80` directly.

The pattern: my agents reported categories of problems where they searched. The auditor specifically searched for known dangerous patterns in known dangerous places. Theirs is a more disciplined methodology.

Finding-by-finding report

H1 — Cross-customer line item RLS leak HIGH

WHAT IT IS

RLS on `invoice_line_items`, `recurring_bill_line_items`, `contract_items`, `quote_items` allows any authenticated user to SELECT all rows. A photography-firm customer login can read a fostering-nonprofit's invoice line items via direct Supabase REST.

WHY I MISSED IT (TECHNICALLY I DIDN'T)

My security agent flagged this as **C2** in its report. I added it to the Tier 1 list but never executed fixes — I asked you which to fix and we got pulled into other work.

IMPACT OF FIXING

- **DB:** Replace `USING (auth.uid() IS NOT NULL)` with a parent-join policy: `EXISTS (SELECT 1 FROM invoices WHERE invoices.id = invoice_line_items.invoice_id AND invoices.customer_id = current_user_customer_id())`
- **App:** No frontend changes — components already fetch parent records first, then line items. RLS just needs to enforce what the code already assumes.
- **Risk if shipped:** Very low. Existing queries continue working because they're already customer-scoped at the parent level.
- **Risk if NOT fixed:** High. Tenant data leak. Any malicious customer user can dump every other tenant's billing data with one fetch.

H2 — Public unauthenticated file proxy HIGH

WHAT IT IS

`GET /api/upload/file/:fileName` proxies the private storage bucket with no auth and `Access-Control-Allow-Origin: *`. Filenames are `Date.now()_<sanitized>.ext` — guessable. SVG is served as `image/svg+xml` (XSS vector).

WHY I MISSED IT (TECHNICALLY I DIDN'T)

My backend agent flagged this as **H3**. Same reason as H1 — I listed it but didn't fix.

IMPACT OF FIXING

- **App breakage:** Any place that embeds files via `/api/upload/file/...` URLs would 401 until the user is authenticated. That includes:
  - Email exports/templates that link to logo/attachment files
  - The customer portal logo display (`CustomerSettings.jsx`)
  - Possibly contract/quote PDF previews
- **Mitigation:** Switch to short-lived signed URLs (5-min expiry) generated server-side and embedded at email-send/page-render time. Existing image tags continue working — URLs just rotate.
- **Risk if shipped:** Medium. Need to test every file-display path. The signed-URL pattern is well-known.
- **Risk if NOT fixed:** Medium-high. File contents leak via guessed/leaked URLs. SVG XSS could escalate to admin compromise if an admin opens a shared link.

H3 — reconciliation_snapshots and reconciliation_excluded_items RLS HIGH

WHAT IT IS

Two LootIT tables grant full read/write to *any* authenticated user. Customers can read other customers' sign-off history, exclusion rules, and billing reconciliation state — and can also tamper with them.

WHY I MISSED IT (TECHNICALLY I DIDN'T)

My backend agent flagged this as **C1 and C2**. Same — listed, not fixed.

IMPACT OF FIXING

- **DB:** Replace `USING (true)` and `USING (auth.uid() IS NOT NULL)` with admin-only writes plus customer-scoped reads (matching the pattern used in `20260331_fix_rls_security.sql`).
- **App breakage:** None. Only LootIT admin users currently write to these tables in normal flow. Customers don't have UI hitting these endpoints.
- **Risk if shipped:** Very low.
- **Risk if NOT fixed:** High. Cross-tenant tampering possible. A customer user could sign off another customer's reconciliation or add exclusions to falsify their billing audit.

H4 — spanning_api_key exposed to customer reads CRITICAL

WHAT IT IS

`spanning_mappings` table stores plaintext `spanning_api_key` (line 411 of `001_initial_schema.sql`). The customer-scoped policy at `001:750–769` includes `spanning_mappings`. **A customer user can fetch their own `spanning_api_key` via Supabase REST.** They can then use it to call Spanning's API directly — read others' backups, change settings, exfiltrate data.

WHY I MISSED THIS (GENUINELY)

I never asked "which columns store credentials, and who can read them?" My agents looked at JS source code for hardcoded keys (none found, correctly), but didn't audit the *schema* for credential columns and cross-reference with RLS. That's the gap.

IMPACT OF FIXING

- **Option A (best):** Drop `spanning_api_key` from `spanning_mappings`. Move credentials to a server-only `vendor_credentials` table that only service-role can read. The backend already uses the service-role client — frontend never needs the key.
- **Option B (faster):** Add a column-level RLS policy that hides `spanning_api_key` from non-admin reads (Supabase supports this via security-invoker views).
- **Option C (cheapest):** Modify the customer-scoped RLS policy to exclude `spanning_mappings` from the customer-readable set entirely.
- **App breakage check needed:** Run `grep -rn "spanning_api_key" src/ apps/lootit/src/` — if frontend code reads this, that path needs server-side proxy.
- **Risk if shipped:** Low if frontend doesn't read it directly (likely the case — backend syncs Spanning).
- **Risk if NOT fixed:** Critical. This is a **vendor credential leak across tenants**. A customer can call Spanning's API impersonating Gamma Tech, with full access to all your Spanning operations.

M5 — RLS audit script doesn't catch all permissive policies MEDIUM

WHAT IT IS

`scripts/test-rls-security.sql` has hardcoded checks that only catch policy *names* matching `Authenticated users can manage`. New permissive policies created with different names slip through (which is exactly what happened with `reconciliation_snapshots`).

WHY I MISSED IT

I read the script's *output* but didn't audit the script's *coverage*. Classic blindspot — the auditor of auditors is missing.

IMPACT OF FIXING

- **Script:** Rewrite to check all `pg_policies` rows for `qual = 'true'` or `qual ILIKE '%auth.uid() IS NOT NULL%'` regardless of policy name. Around 10 lines.
- **CI:** Run this on every PR; fail the build if any tenant table has a permissive policy.
- **App breakage:** None — internal tooling.
- **Risk if shipped:** None.
- **Risk if NOT fixed:** Future RLS regressions silently pass. Auditor proves this isn't theoretical.

M6 — Login open redirect MEDIUM

WHAT IT IS

`Login.jsx:35` reads `?returnUrl=` from the query string and `Login.jsx:80` does `window.location.href = returnUrl` after successful login. No same-origin check. An attacker sends `https://portalit.gtools.io/login?returnUrl=https://evil.com/phishing` — user logs in, gets redirected to attacker site.

WHY I MISSED IT

My security agent flagged "open redirect" as a category to watch but didn't grep specifically for `returnUrl|window.location.href` patterns. Generic vs. specific.

IMPACT OF FIXING

- **Code:** One-line guard:

```
if (returnUrl && returnUrl.startsWith('/') && !returnUrl.startsWith('//')) {
  window.location.href = returnUrl;
}
```

- **App breakage:** None — legitimate use is always relative paths.
- **Risk if shipped:** Zero.
- **Risk if NOT fixed:** Phishing vector. Attacker dresses up a portalit.gtools.io login link to steal credentials by redirecting to a clone after success.

M7 — Auth tokens in URL fragment MEDIUM

WHAT IT IS

Login flow at `Login.jsx:73-74` redirects to `${CUSTOMER_PORTAL_URL}/auth-redirect#access_token=...&refresh_token=...`. URL fragments leak via:

- Browser bookmarks
- Browser history sync (Chrome → Google account)
- Password manager URL capture
- Some referrer-leak edge cases

WHY I MISSED IT (TECHNICALLY I DIDN'T)

My security agent flagged this as **H-6**. Listed, not fixed.

IMPACT OF FIXING

- **Cleanest:** Remove the fragment-handoff entirely. The `.gtools.io` cookie already carries the session across subdomains (via `auth-storage.js`). Just redirect to `${CUSTOMER_PORTAL_URL}/`.
- **App breakage:** Need to verify the cookie flow works on customer portal subdomain. If the cookie is set on `portalit.gtools.io` only (not `.gtools.io`), customer portal won't see it and login will silently fail.
- **Risk if shipped:** Medium — auth flow changes deserve real testing on multiple browsers.
- **Risk if NOT fixed:** Tokens leak. Refresh tokens are long-lived, so a leaked refresh token = persistent access until rotated.

M8 — INKY token storage in browser extensions MEDIUM

WHAT IT IS

`extensions/lootit-link/inject.js` and `extensions/inky-sync/*` intercept INKY bearer/refresh tokens and store them in `chrome.storage.local`. If the extension is installed by anyone outside your team, those tokens are in their browser cache.

WHY I MISSED IT

I excluded `extensions/` from agent scope — didn't even know it existed.

IMPACT OF FIXING

- **Scope:** First, audit who has these extensions installed (just Gamma techs, or also customers?). If customer-installed, this is significantly worse.
- **Mitigation:** Use only access tokens (short-lived), don't store refresh tokens client-side, scope token use to the active session, document clearly that extension is internal.
- **App breakage:** Depends on what features need the refresh token persisted. If "auto-resume after browser restart" is a feature, you'd lose it.
- **Risk if NOT fixed:** Medium. INKY token theft = read others' email phishing analyses. Privacy issue more than active-attack vector.

M9 — npm audit vulns MEDIUM

WHAT IT IS

Root: 13 prod findings (high in `react-router-dom`, `lodash`, `glob/minimatch/picomatch`). Server: 3 prod findings (high in `path-to-regexp` via Express, moderate in `node-cron/uuid`).

WHY I MISSED IT (TECHNICALLY I DIDN'T)

Security agent flagged these as **M-11/M-12**. Listed, not fixed.

IMPACT OF FIXING

- **lodash upgrade:** Drop-in. Zero breakage expected.
- **react-router-dom upgrade:** Could be a major version bump. v6 → v7 has API changes (Routes/Outlet behavior). Needs testing on all routes.
- **path-to-regexp via Express:** Bump to Express ≥4.20 or pin transitively. Test all route patterns since regex behavior could shift.
- **App breakage risk:** Medium — touches routing in two places (Express + React Router). Run E2E tests after.
- **Risk if NOT fixed:** Each CVE has different exploitability. `path-to-regexp` DoS via crafted route patterns is the most exploitable in production.

M10 — lootit_contracts table and bucket schema drift MEDIUM

WHAT IT IS

Frontend uses `lootit_contracts` table and `lootit-contracts` storage bucket. They exist in production (verified) but no migration creates them. They were created by hand via Supabase dashboard.

WHY I MISSED IT

I didn't cross-reference `client.entities.*` calls against `supabase/migrations/` to find tables that exist in code but not in schema files.

IMPACT OF FIXING

- **Generate retrospective migration:** Dump current schema for `lootit_contracts` table and `lootit-contracts` bucket policies, write a migration file. Mark it as already-applied in production.
- **App breakage:** None — just makes existing reality reproducible.
- **Risk if shipped:** None.
- **Risk if NOT fixed:** A new environment (staging, future tenant) won't have these resources, app breaks silently. Schema-drift is a compounding cost.

Risk-vs-effort matrix

Finding	Impact if exploited	Fix effort	Test surface
H4 spanning_api_key	**Critical — vendor credential leak**	Small (drop column or restrict policy)	Server sync still works
H1 line items RLS	**High — billing data leak**	Small (replace policy)	None (frontend already scoped)
H3 reconciliation tables RLS	**High — cross-tenant tampering**	Small (replace policy)	None
H2 file proxy	**Medium — file/SVG leak**	Medium (signed URLs)	All file display paths
M7 fragment tokens	**Medium**	Medium (cookie handoff)	Auth flow on multiple browsers
M9 npm vulns	**Medium (CVE-dependent)**	Medium-large (router upgrade)	All routes
M6 open redirect	Low (phishing)	Trivial (1 line)	None
M5 audit script	None now, future regressions	Small (rewrite)	None
M8 extension tokens	Low (internal)	Medium (token redesign)	Extension flows
M10 schema drift	None now, future env breaks	Small (write migration)	None

Recommended fix order

Wave 1 (today, low-risk wins): H4 spanning_api_key + H1 line items + H3 reconciliation tables + M5 audit script + M6 open redirect + M10 schema drift. All RLS-only, no app behavior changes, ~30 min of SQL + 1 line of JS + 1 SQL script.

Wave 2 (this week, needs testing): H2 file proxy (signed URLs) + M7 cookie handoff. Touches auth + file display, deserves QA.

Wave 3 (when ready): M9 dependency upgrades + M8 extension token cleanup. Major-version bumps need bandwidth for regression testing.